



BioCypher KG Console

The Support Folders, Explained

test · scripts · biocypher_metta · biocypher-mork · logs
kg-service · notebooks · GitHub workflows

Prepared for: Biruk · HabEX Creative Agency

Table of Contents

1 Why This Guide

The main Console (frontend + backend) is only part of the repository. Around it sit eight support folders that make the project trustworthy, runnable, explorable, and automatically checked. Each one exists to answer a different practical question. This guide covers each folder's job, its most important files, and — where it matters — exactly why it was built the way it was.

Folder	The one question it answers
test/	"Did this change break anything?"
scripts/	"How do I run a one-off operational task?"
biocypher_metta/	"How does raw data actually become graph facts?"
biocypher-mork/	"Where does the graph live if not in Neo4j?"
biocypher-log/ (generated)	"What exactly happened during this build?"
kg-service/	"How does a person use all of this without a terminal?"
notebooks/	"How do researchers explore data before it becomes production code?"
.github/workflows/	"What gets checked automatically, without a human remembering to run it?"



test/

Automated checks that catch mistakes before they reach real users

2.1 What it is

A collection of automated test files, run by a tool called **pytest**. Each file checks that one part of the system behaves correctly — without a human needing to click through the website by hand every time something changes.

2.2 What's inside

File	What it checks
conftest.py	Shared setup every other test file relies on — makes sure Python can find the project's code, and redirects the Console's build/job storage to a temporary folder so tests never touch real build data.
test.py	The core pipeline smoke test — runs BioCypher against sample data through real adapters, confirming the whole read → schema → assemble chain still works end to end.
test_console_builds.py	Confirms the Console's build-job system (create, track, cancel a job) behaves correctly.
test_console_config.py	Confirms the Console correctly reads species/dataset/adapters/schema configuration.
test_console_validation.py	Confirms validate_build() correctly catches bad build requests before they run.
test_neo4j_version_finalize.py	Confirms dataset version bookkeeping is finalized correctly after a Neo4j load.
test_provenance.py	Confirms every graph fact correctly records which data source and adapter produced it.
test_versioning.py	Confirms the dataset-version tracking system behaves correctly as data sources update.

2.3 Why it matters

- Without this folder, every code change would need to be checked by hand, by re-running a full build and visually inspecting the result — slow, and easy to get wrong.
- This is also the folder the GitHub workflows (Section 9) run automatically on every change — the two folders work as a pair.



scripts/

Stand-alone operational tools, run one at a time, by hand, for a specific task

3.1 What it is

A toolbox of independent Python and shell scripts. Unlike the Console (which is always running, waiting for clicks) or the pipeline (which runs the same way every time), each script here is started manually, does one specific job, and exits. Think of it as the project's "maintenance drawer."

3.2 What's inside, grouped by purpose

Loading data into a database

- `neo4j_loader.py` — imports finished graph output into a Neo4j database.
- `mork_loader.py` — the interactive loader for MORK, prompting for a dataset path and whether to clear existing data first.
- `mork_client.py` / `mork_repl.py` — a client library for talking to MORK, and a REPL (a live, type-and-run console) for poking at it directly.

Build automation

- `build.sh` — a Docker-oriented script: copies the project, installs dependencies, runs the full knowledge-graph build, then makes the resulting `biocypher-log` folder world-readable.
- `build_act.sh` / `build_paths.sh` — variants of the build process used for local (`act` = GitHub Actions run locally) or path-specific runs.
- `import.sh` — waits for a Neo4j admin-import script to appear, then starts and stops Neo4j once so the import takes effect.

Data & ontology mapping helpers

- Several scripts (e.g. `create_catlas_cell_ontology_map.py`, `dmel_create_entrez_to_ensembl_map.py`, `update_dbsnp.py`) each build or refresh one specific lookup table a particular adapter needs — for example, mapping one gene-ID system to another.

Everyday utilities

- `pkl_cat.py` — prints the contents of a `.pkl` (Python "pickle") cache file, for quick inspection.
- `sort_schemas.py` / `reorganize_schemas.py` — housekeeping for keeping schema YAML files tidy and consistently ordered.

3.3 Why a separate folder instead of adding these to the main pipeline

Each of these is run rarely, by a developer, on purpose — not automatically every time the pipeline runs. Keeping them separate means the main build stays fast and simple, while these occasional tools stay easy to find, run, and update independently.



4.1 What it is

This is the heart of the data pipeline described in the earlier Processing Guide. It holds every **adapter** (one per data source) and every **writer** (one per output format).

4.2 What's inside

Sub-part	Role
adapters/ (46 files)	One file per raw data source (genes, variants, ontologies, protein interactions, and more) — each knows how to read that one source and emit graph facts.
processors/	Shared helper logic used by several adapters at once — for example, mapping between different gene ID systems (HGNC, Ensembl, Entrez) so adapters don't each reimplement the same lookup.
neo4j_csv_writer.py / neo4j_writer.py	Write the finished graph in formats Neo4j can import directly.
metta_writer.py	Writes the graph in MeTTa format — the format MORK (Section 5) actually loads.
parquet_writer.py	Writes the graph as Parquet files — a compact, fast, table-based format useful for data analysis outside a graph database.
kgx_writer.py	Writes in KGX format, a community standard for exchanging knowledge graphs with other tools/projects.
networkx_writer.py	Writes the graph so it can be loaded directly into NetworkX for in-Python analysis.
prolog_writer.py	Writes the graph as Prolog facts, for logic-programming-based querying/reasoning.
provenance.py	Tracks, for every fact, exactly which data source and adapter produced it — this is what test_provenance.py checks.

4.3 Why so many writers for one graph

The same finished graph is useful in different forms to different tools — a graph database wants CSV/Neo4j format, an analyst wants Parquet or NetworkX, and MORK wants MeTTa. Rather than forcing one format on everyone, the project builds the graph once and can export it multiple ways.



5.1 What it is

A self-contained sub-project (with its own Dockerfile and README) that runs **MORK** — a lightweight graph/atomspace server — and loads BioCypher's MeTTa-format output into it, as an alternative or companion to Neo4j.

5.2 What's inside

File / folder	Role
Dockerfile / docker-compose.yml	Packages and runs the MORK server as a container.
data_loader.py	Loads MeTTa-format graph data into the running MORK server.
client.py / wal_client.py	Python clients for talking to MORK — the second one specifically speaks to its Write-Ahead Log (WAL) system.
mork_persist/ (wal.metta, snapshot.paths)	Where MORK keeps its crash-safe persistence — a Write-Ahead Log plus periodic snapshots, so data survives a restart.
.env.example	Template for the one setting that matters most: MORK_DATA_DIR, which points MORK at BioCypher's output folder.

5.3 Why crash-safe persistence matters here specifically

MORK is designed to hold the graph largely in memory for speed. Without the Write-Ahead Log and snapshot system, a server crash or restart could lose data that was never written to disk. The WAL records every change as it happens, so the graph can always be rebuilt to its last known-good state.

5.4 Why the project offers MORK as well as Neo4j

Neo4j is a mature, general-purpose graph database with rich query tools. MORK is lighter-weight and integrates with the OpenCog/Hyperon ecosystem the wider project is connected to (the project's own description mentions Hyperon and Prolog as targets). Offering both means the same knowledge graph can serve two different downstream use cases without building it twice.



biocypher-log/ (generated automatically)

The pipeline's own record of exactly what happened during a run

6.1 What it is

This folder is not something you'll find already sitting in the repository — it is created automatically by the BioCypher library itself the moment a build runs, in the same folder the build was started from. It is the pipeline's own diary of that run.

6.2 Where it's confirmed in this project

The project's own Docker build script makes this explicit — after running the pipeline, it runs `chmod -R 777 biocypher-log` specifically so the generated log folder is readable by whoever needs to inspect it afterward.

6.3 What ends up in it

- Which adapters ran, in what order, and how long each took.
- Warnings about malformed or unexpected rows in a raw data file.
- Schema mismatches — a fact that didn't fit any allowed type or relationship.
- The final summary of how many nodes and edges were produced.

6.4 How this differs from the Console's own logs

The Console (in [kg-service/](#)) keeps its own, separate log stream per build job — captured live and shown in the browser, and stored under its `.builds/` folder for later lookup. `biocypher-log/` is a level deeper: it's the pipeline engine's own internal diary, produced regardless of whether the Console or a plain terminal command triggered the run.



kg-service/

The Console — the friendly front door to everything above

7.1 What it is

Covered in depth in the earlier project guides — this is the web application (FastAPI backend + React frontend) that lets a person configure, launch, watch, and review a knowledge-graph build without touching a terminal, and that Phase 2 is extending with query, exploration, and config-editing features.

7.2 How it relates to every folder around it

kg-service/ calls...	...to do this
biocypher_metta/ + create_knowledge_graph.py	Run the actual pipeline as a background job, when a build is launched.
scripts/neo4j_loader.py, scripts/mork_loader.py logic	Load a finished build's output into Neo4j and/or MORK.
biocypher-mork/	The MORK server a load can target, if MORK is selected as a backend.
biocypher-log/ (generated)	An additional, lower-level source of detail if a build's Console-level logs aren't enough to diagnose an issue.
test/test_console_*.py	The safety net that confirms kg-service/ itself keeps working correctly as it changes.

In short: every other folder in this guide does real work; [kg-service/](#) is the layer that makes that work usable by a person, safely and visually.



notebooks/

Interactive research scratchpads — exploration, not production

8.1 What it is

Three Jupyter notebooks — an interactive, cell-by-cell coding environment popular in data science and research, where you can run one small piece of code, immediately see its output (a table, a chart), and then decide what to try next, without re-running the whole file each time.

8.2 What's inside

Notebook	What it explores
cellxgene_exp_corr.ipynb	Correlating gene expression data between two sources (GTEx bulk tissue data and CellxGene single-cell data) for a specific tissue/cell type — exploratory analysis, with data visualizations built up interactively.
finemapping_susie.ipynb	Statistical fine-mapping of GWAS (genome-wide association study) summary statistics using the SuSiE method, to narrow down likely causal genetic variants.
llm_connect.ipynb	Experimenting with connecting a local LLM (via Ollama/LangChain) to interpret biological enrichment terms in plain language.

8.3 Why use a notebook environment for this, specifically

- **Immediate visual feedback.** Research questions like "is this gene's expression correlated between two datasets" are answered by looking at a plot, not just a return value — a notebook shows that plot inline, right next to the code that made it.
- **Cell-by-cell iteration.** Fine-mapping and correlation analysis are naturally exploratory: load data in one cell, check it looks right, then try three different parameter choices in the next cells without re-loading the data each time. A script would rerun everything from scratch on every change; a notebook doesn't.
- **Keeps research separate from production.** None of this exploratory code needs to be reliable, tested, or fast — it needs to be flexible while someone is still figuring out the right analysis. Once an approach is proven here, the **stable, reusable** parts of it graduate into a real adapter or processor in [biocypher_metta/](#) — the notebook is the sketchpad, not the final artifact.
- **Matches how the wider field already works.** Jupyter is the standard tool computational biologists and bioinformaticians already use daily — using it here means a researcher on the team can contribute analysis without needing to learn the Console's or the pipeline's software engineering conventions first.



.github/workflows/

Automatic checks that run themselves — no human has to remember to

9.1 What it is

A set of GitHub Actions definitions — automation rules that GitHub itself runs, triggered by events like "someone pushed code" or "it's Monday morning," without anyone needing to be at a keyboard.

9.2 What's inside

Workflow file	What it automates, and when it runs
test-adapters.yml	Runs the adapter/writer tests whenever adapter or writer code changes — catches a broken adapter before it's merged.
test-schema.yml	Runs adapters against the schema rulebook whenever schema or adapter files change — catches a mismatch between what an adapter produces and what the schema allows.
check-dataset-versions.yml	Runs every Monday morning (and on demand) — checks whether any upstream data source has published a new version, flagging drift so stale data doesn't go unnoticed.
ci-quality-analysis.yml	Runs a SonarQube code-quality scan on every push/pull request to the main branches — flags code smells, duplication, and potential bugs automatically.
dependabot-auto-merge.yml	Automatically approves and merges dependency-update pull requests opened by Dependabot, when they meet the criteria — keeps libraries current with less manual review overhead.

9.3 Why automate these specifically

- **Adapters and schema** change often (46 adapters!) — a human reviewer can't realistically re-run the full test suite by hand for every pull request; the workflow does it every time, automatically.
- **Dataset versions** drift silently — nobody manually re-checks 40+ external data sources every week, so a scheduled job does it instead.
- **Code quality and dependency freshness** are easy to deprioritize under deadline pressure — automating them means they happen regardless of how busy the team is.

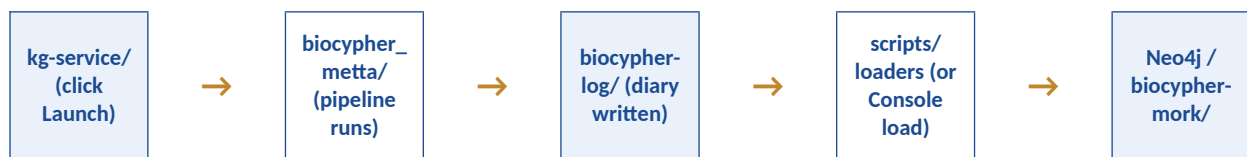
10 How All Eight Folders Connect

A simplified view of how a piece of work moves between these folders in practice.

10.1 A researcher's exploratory path



10.2 A user's operational path



In one paragraph

notebooks/ is where new analysis ideas are tried out; proven ideas graduate into real adapters inside biocypher_metta/, protected by test/ and automatically checked by .github/workflows/. scripts/ holds the operational tools for loading and maintaining data outside the main pipeline. Every real build writes its own diary to biocypher-log/. The finished graph can live in Neo4j or in biocypher-mork/'s crash-safe MORK server. And kg-service/ is the friendly front door that lets a person trigger and watch all of this without touching a terminal.